Lab 4: Memory Management
Geavanceerde Computerarchitectuur

Sien Peerlinck & Thibaut Dalez

March 22, 2024

1 Introduction

This was a lab on memory management consisting of two parts. The first part was on coalesced versus uncoalesced memory access, the second part was on global, shared and constant memory.

In coalesced memory access, consecutive threads within a warp access consecutive memory locations. When threads within a warp access memory in this way, the GPU can efficiently combine these individual memory transactions into larger, more efficient transactions. This reduces the total number of memory transactions required to fulfill the memory access requests of the warp, thus improving memory bandwidth utilization and overall performance

Uncoalesced memory access occurs when consecutive threads within a warp access non-consecutive memory locations. this way, each thread's memory request may result in a separate memory transaction, which leads to reduced memory bandwidth utilization and can degrade overall performance compared to coalesced memory access.

The global memory resides off-chip and is accessible by all threads within a GPU kernel and persists for the duration of the application. It's suitable for storing large amounts of data that need to be accessed by multiple threads, but accessing global memory can be slower compared to other types of memory due to its off-chip location and higher latency.

Shared memory is located on the GPU chip itself, it is a small but very fast type of memory. It is shared among threads within a block and has a lower latency compared to global memory.

Constant memory is a small type of memory that resides in the global memory space, but is cached on-chip for faster read access.

2 Part 1: coalesced vs. uncoalesced memory access

The task for this part of the lab was to design two kernels that invert only the red color components of an image. One kernel had to do it in an uncoalesced way, the other coalesced. For both implementations runtime was measured for a number of threads ranging from 0 to 1023, the average runtime was taken over 20 runs for each number of threads. The timing results can be seen in figure 1. The graph shows an increase in speed if more threads are allocated, which seems right, but it also shows only a small difference in timing between coalesced and uncoalesced memory access and coalesced is slower than uncoalesced.

3 Part 2: Global, shared and constant memory

The task for this part of the lab was to multiply two matrices using different techniques for memory accessing. For each technique runtime was assessed for matrix sizes 1 to 100, average runtime was taken over 100 runs. Results for this can be seen in figure 2

The first technique we used uses global memory to calculate the matrix operations. This technique has sufficient memory to do all the calculations and save the matrices so no techniques are needed here to split up the matrices. The code for the multiplication of the matrices are in "matrixmultiply", here the row and column are defined so we can correctly iterate over each matrix in the threads, the result of the multiplication is put into "tmpsum". Then a syncthreads is used so we know for sure that each multiplication will be put in the correct position of matrix C.

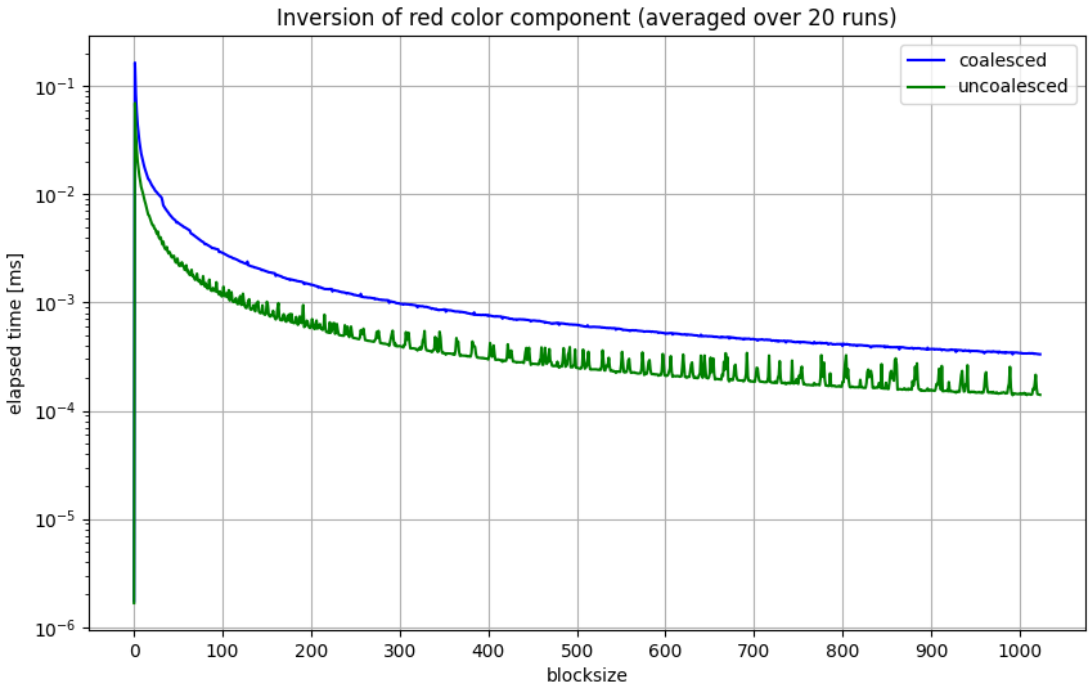


Figure 1: Timing results for inverting the red color components with coalesced (blue) and uncoalesced (green) memory access

Shared memory has a higher bandwidth and lower latency then local or global memory. This is because shared memory is on-chip and consists of memory modules that can be accessed simultaneously. As seen in figure 2 this is the most efficient. Because shared memory has a max capacity of 48kByte, we only copy the part of the matrix that we need at the moment int[illegible] shared memory. This way we can compute matrices that would take more than 48kByte to compute (fe, in global memory). When the matrix is bigger then the blocksize (32) we see that the elapsed time rises sharply because it can't be calculated in one time anymore. Because of this you can see that the shared memory timing scales with the matrix size.

Constant memory is cashed in the constant cache which is only 64kB, it is optimised for low latency read operations. We need to define the memory beforehand, it is not dynamic like the other memories. Because of this we defined it with a big capacity (78*78, the max size is 89*89) so we could test different sized matrices in 1 time. Except for the use of constant matrices, the way of calculating the matrices are the same as with global memory. Because of the matrix size limitations we have we only could let it run till 78, in figure 2 the matrices are bigger then 78, but after 78 the matrix C that came from the calculations was wrong. We did those calculations because we where curious on how the timing would evolve.

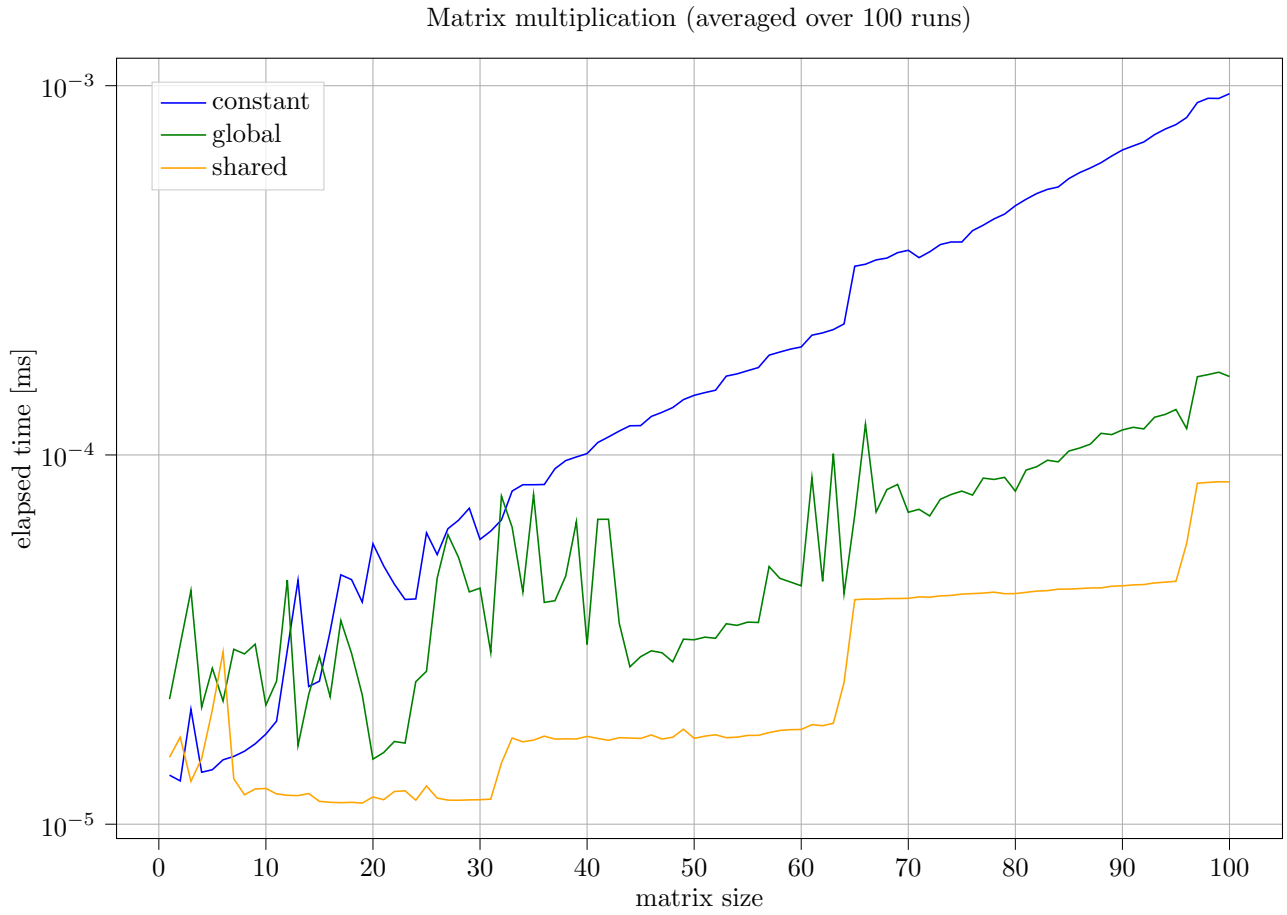


Figure 2: Timing results for matrix multiplication with matrix sizes upto 100 for constant, global and shared memory access.

A Program

The code can be found at `https://github.com/SienPeerlinck/Geavanceerde-Computerarchitectuur/blob/16b47e29ce4c47f48aaf57ba66d9bead1b77a22c/lab4b3.cu`

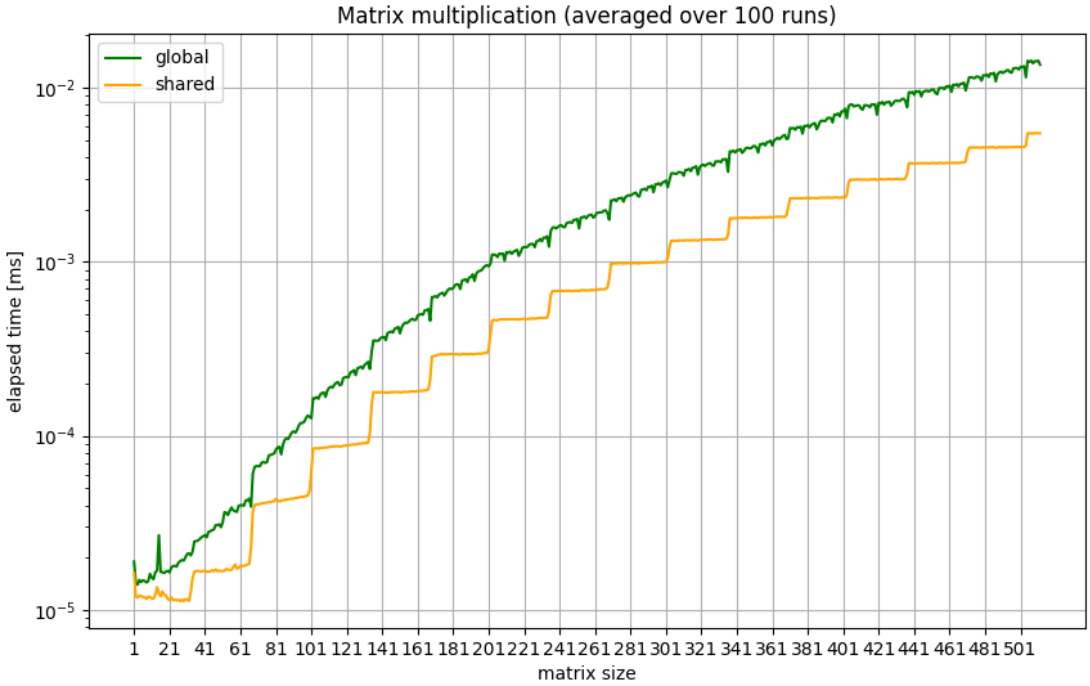


Figure 3: Timing results for global and shared memory access for matrix sizes upto 512.